SQL Subquery — Practice Question Set

Table Setup (copy-paste into MySQL)

```
CREATE DATABASE IF NOT EXISTS subquery_practice;
USE subquery_practice;

CREATE TABLE movies (
    movie_id INTEGER PRIMARY KEY AUTO_INCREMENT,
    name VARCHAR(255) NOT NULL,
    rating VARCHAR(10),
    genre VARCHAR(50),
    release_year INTEGER,
    released DATE,
    score DECIMAL(3,1),
    votes INTEGER,
    director VARCHAR(100),
    writer VARCHAR(100),
    star VARCHAR(100),
    country VARCHAR(100),
    budget DECIMAL(15,2),
    gross DECIMAL(15,2),
    company VARCHAR(100),
    runtime DECIMAL(5,1)
);

CREATE TABLE users (
    user_id INTEGER PRIMARY KEY AUTO_INCREMENT,
    name VARCHAR(100) NOT NULL,
    email VARCHAR(150) NOT NULL UNIQUE,
    password VARCHAR(255)
);

CREATE TABLE orders (
    order_id INTEGER PRIMARY KEY AUTO_INCREMENT,
    user_id INTEGER NOT NULL,
    r_id INTEGER,
    amount DECIMAL(10,2),
    restaurant_rating DECIMAL(3,1),
    order_date DATETIME DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT orders_fk FOREIGN KEY (user_id) REFERENCES users(user_id)
);

CREATE TABLE order_details (
    order_detail_id INTEGER PRIMARY KEY AUTO_INCREMENT,
    order_id INTEGER NOT NULL,
    f_id INTEGER NOT NULL
);

CREATE TABLE food (
    f_id INTEGER PRIMARY KEY AUTO_INCREMENT,
    f_name VARCHAR(100) NOT NULL
);
```

```
CREATE TABLE restaurants (
    r_id INTEGER PRIMARY KEY AUTO_INCREMENT,
    r_name VARCHAR(100) NOT NULL
);

CREATE TABLE loyal_users (
    user_id INTEGER PRIMARY KEY,
    name VARCHAR(100),
    money DECIMAL(10,2)
);

CREATE TABLE delivery_partner (
    partner_id INTEGER PRIMARY KEY AUTO_INCREMENT,
    name VARCHAR(100)
);
```

Topic 1: What is a Subquery / Basics

1. Define a subquery in your own words and identify the inner and outer query in:

`SELECT * FROM movies WHERE score = (SELECT MAX(score) FROM movies);`

2. Find the movie with the highest rating (using a subquery, not ORDER BY).

Topic 2: Independent Subquery — Scalar (single value)

3. Find the movie with highest profit (gross - budget), using a subquery — compare with the ORDER BY approach.

4. Find how many movies have a rating > the avg of all the movie ratings (find the count of above-average movies).

5. Find the highest rated movie of the year 2000.

6. Find the highest rated movie among all movies whose number of votes is greater than the dataset's average votes.

7. Find the difference from average rating for all Samsung phone ratings (use ABS) — *(smartphones dataset)*.

Topic 3: Independent Subquery — Row (one column, multiple rows)

8. Find all users who never placed an order.

9. Find all the movies made by the top 3 directors (in terms of total gross income).

10. Find all movies of all those actors whose filmography's avg rating > 8.5 (take 25000 votes as cutoff).

Topic 4: Independent Subquery — Table (multiple columns, multiple rows)

11. Find the most profitable movie of each year.

12. Find the highest rated movie of each genre (votes cutoff of 25000).

13. Find the highest grossing movies of the top 5 actor/director combos (in terms of total gross income).

Topic 5: Correlated Subquery

14. Find all the movies that have a rating higher than the average rating of movies in the same genre (e.g., Animation).

15. Find the favorite (most ordered) food of each customer.

Topic 6: Usage with SELECT

16. Get the percentage of votes for each movie compared to the total number of votes (across all movies).

17. Display all movie names, genre, score, and the average score of that movie's genre — all in the same row. Why is this approach inefficient on large tables?

Topic 7: Usage with FROM

18. Find the average restaurant rating per restaurant (using orders), then join it back with the restaurants table to display restaurant name and average rating.

19. Find each user's favorite food (highest order frequency) by first building a frequency table in a subquery, then selecting from it.

Topic 8: Usage with HAVING

20. Find genres having an average score greater than the average score of all the movies.

21. Find the avg rating of smartphone brands which have more than 20 phones — *(smartphones dataset, but solve using HAVING + subquery)*.

Topic 9: Subquery in INSERT

22. Populate an already-created `loyal_users` table with records of only those customers who have ordered food more than 3 times.

Topic 10: Subquery in UPDATE

23. Populate the `money` column of the `loyal_users` table using the `orders` table — provide 10% of a customer's total order value as loyalty money.

Topic 11: Subquery in DELETE

24. Delete all customer records from `users` who have never placed an order.

Topic 12: Mixed / Challenge

25. Find the top 3 brands with the highest avg RAM that have a refresh rate of at least 90Hz and fast charging available — don't consider brands with fewer than 10 phones *(combine HAVING + filtering, no subquery needed — identify why)*.

26. Find the avg price of all phone brands with avg rating > 70 and more than 10 phones, among all 5G-enabled phones.

27. Write a query to find the movie(s) whose votes are above the dataset's average votes AND whose score is the highest among that filtered group (combine two subquery concepts in one query).

28. Find all restaurants whose average rating is higher than the platform-wide average restaurant rating (correlated or independent — your choice, justify which one you used).